Deber Estructura de datos

I. PORTADA

Tema:	ARBOLES
Unidad de Organización Curricular:	BÁSICA
Nivel y Paralelo:	Tercero B
Alumnos participantes:	Nata Analuiza Walter Alexis
Asignatura:	Estructura de Datos
Docente:	Ing. Jose Caiza, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

- Implementar en C++ un Árbol Binario de Búsqueda, BST para organizar empleados de una empresa usando un código numérico como clave. El ejercicio se relaciona con la diapositiva del organigrama empresarial, donde se identifican raíz, niveles, nodos internos y hojas.

Específicos:

- Diseñar la estructura de datos del nodo para representar a los empleados, asegurando la correcta asignación de roles mediante la identificación técnica de la raíz (gerencia), nodos internos (supervisores) y hojas (operativos) dentro del árbol.
- Desarrollar las funciones lógicas de inserción y búsqueda basadas en el código numérico del empleado, garantizando que el árbol mantenga sus propiedades de ordenamiento para optimizar los tiempos de respuesta en la localización de registros.
- Codificar métodos que permitan calcular y visualizar la profundidad de los niveles y el recorrido del organigrama.

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 4
No presenciales: 0

2.4 Instrucciones

1. Revisar el material observado en clases.
2. Investigar sobre el tema de arboles y como se los aplica.
3. Diseñe la solución.
4. Implemente la solución propuesta.
5. Elabore un informe del trabajo realizado.
6. Suba el informe a la plataforma.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial, TAC

- Computador, Cuadernos, esferos, lápiz, borrador, Aula virtual e Internet

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

☒Plataformas educativas
☒Simuladores y laboratorios virtuales
☒Aplicaciones educativas
☐Recursos audiovisuales
☐Gamificación
☒Inteligencia Artificial
Otros (Especifique): _____

2.6 Actividades por desarrollar

Una empresa necesita registrar empleados en una estructura jerárquica usando un árbol binario de búsqueda.

- **Cada empleado tendrá:**
  - codigo
  - nombre
  - cargo

- **El árbol debe permitir:**
  - Insertar empleados.
  - Buscar un empleado por código.
  - Mostrar recorrido inorden.
  - Mostrar recorrido preorden.
  - Mostrar recorrido postorden.
  - Calcular altura del árbol.
  - Identificar hojas.
  - Mostrar la raíz del árbol.

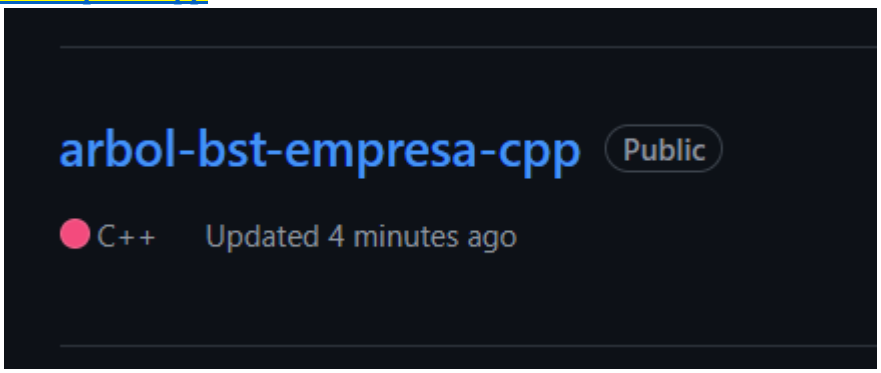
- **Organización del grupo**
- **Rol Responsabilidad**
  - Líder Coordina el avance y revisa el repositorio
  - Programador 1 Implementa estructura Nodo y función insertar
  - Programador 2 Implementa búsqueda y recorridos
  - Programador 3 Implementa altura y hojas
  - Documentador Crea README, capturas y explicación
  - Expositor Presenta funcionamiento y conclusiones

2.7 Resultados obtenidos

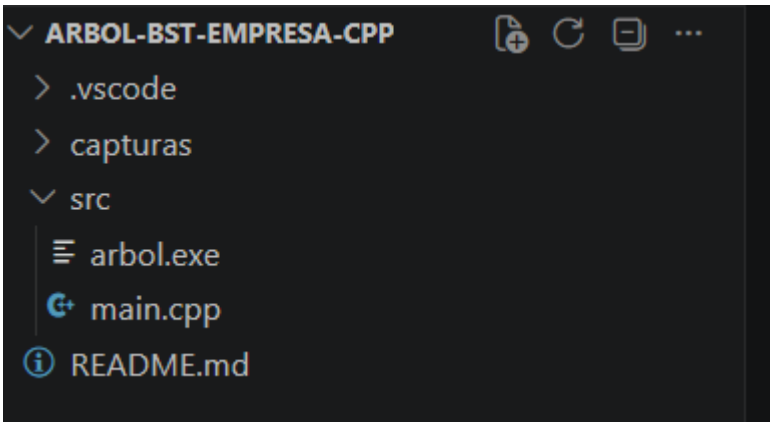
Conceptos Claves:

- Raíz: El nodo raíz es el punto de inicio de un árbol, siendo el único nodo en la parte superior. Desde aquí, se pueden realizar diversas operaciones, como búsquedas y recorridos en la estructura del árbol.[1]
- Nodo interno: Nodo con al menos un hijo. [2]
- Hoja: Es un nodo sin hijos. [2]
- Nivel: Es el numero de aristas entre un nodo y la raíz +1. [2]
- Altura: Numero de aristas en el camino mas largo de un nodo y una hoja descendiente. [2], [3]

1. Creamos el repositorio con el nombre : arbol-bst-empresa-cpp



Estructura de la tarea:



En la carpeta captura colocaremos las capturas de pantalla en la que reflejaremos la ejecución del código.
Crearemos una carpeta /src en la cual estará nuestro main.cpp el cual contiene el código fuente.
En el proyecto también se ubicará el archivo README.md.

Código Fuente:

```
struct Empleado {
    int codigo;
    string nombre;
    string cargo;
};
```

Define los datos que almacena cada nodo del árbol: un código numérico, el nombre del empleado y su cargo dentro de la empresa.

```
struct Nodo {
    Empleado dato;
    Nodo* izquierdo;
    Nodo* derecho;

    Nodo(Empleado emp) {
        dato = emp;
        izquierdo = nullptr;
        derecho = nullptr;
    }
};
```

Representa cada nodo del árbol. Contiene un objeto Empleado como dato, y dos punteros: izquierdo y derecho, que apuntan a sus nodos hijos. Al crearse, ambos punteros se inicializan en nullptr.

```
class ArbolBST {
private:
    Nodo* raiz;

    Nodo* insertar(Nodo* nodo, Empleado emp) {
        if (nodo == nullptr) {
            return new Nodo(emp);
        }

        if (emp.codigo < nodo->dato.codigo) {
            nodo->izquierdo = insertar(nodo->izquierdo, emp);
        } else if (emp.codigo > nodo->dato.codigo) {
            nodo->derecho = insertar(nodo->derecho, emp);
        } else {
            cout << "El codigo ya existe.\n";
        }

        return nodo;
    }
```

Es el núcleo del programa. Contiene un puntero privado raíz con varias funciones como la siguiente de insertar(). Compara el código del empleado con el nodo actual. Si es menor va a la izquierda, si es mayor va a la derecha, hasta encontrar un espacio vacío.

```
Nodo* buscar(Nodo* nodo, int codigo) {
    if (nodo == nullptr || nodo->dato.codigo == codigo) {
        return nodo;
    }

    if (codigo < nodo->dato.codigo) {
        return buscar(nodo->izquierdo, codigo);
    } else {
        return buscar(nodo->derecho, codigo);
    }
}
```

Función buscar(). Recorre el árbol comparando códigos de forma recursiva hasta encontrar el empleado o llegar a un nodo nulo.

```
void inorden(Nodo* nodo) {
    if (nodo != nullptr) {
        inorden(nodo->izquierdo);
        mostrarEmpleado(nodo);
        inorden(nodo->derecho);
    }
}

void preorden(Nodo* nodo) {
    if (nodo != nullptr) {
        mostrarEmpleado(nodo);
        preorden(nodo->izquierdo);
        preorden(nodo->derecho);
    }
}

void postorden(Nodo* nodo) {
    if (nodo != nullptr) {
        postorden(nodo->izquierdo);
        postorden(nodo->derecho);
        mostrarEmpleado(nodo);
    }
}
```

Recorridos:
Inorden(): Recorre izquierda → raíz → derecha. Muestra los empleados ordenados de menor a mayor código.
Preorden(): Recorre raíz → izquierda → derecha. Empieza desde la raíz y desciende.
Postorden(): Recorre izquierda → derecha → raíz. Los nodos hoja aparecen primero.

```
int altura(Nodo* nodo) {
    if (nodo == nullptr) {
        return 0;
    }

    int alturaIzq = altura(nodo->izquierdo);
    int alturaDer = altura(nodo->derecho);

    return 1 + max(alturaIzq, alturaDer);
}
```

Altura(), calcula recursivamente la altura del árbol comparando el nivel izquierdo y derecho, retornando el mayor + 1.

```
void mostrarHojas(Nodo* nodo) {
    if (nodo != nullptr) {
        if (nodo->izquierdo == nullptr && nodo->derecho == nullptr) {
            mostrarEmpleado(nodo);
        }
        mostrarHojas(nodo->izquierdo);
        mostrarHojas(nodo->derecho);
    }
}
```

mostrarHojas(). Identifica nodos que no tienen hijos (izquierdo y derecho en nullptr).

Capturas de la Ejecución:

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 1
```

Se muestra el menú con todas las opciones disponibles del sistema BST.

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 1
Codigo: 50
Nombre: Empresa UTA
Cargo: Raiz
```

Se insertaron 7 empleados con sus códigos, nombres y cargos. El primer empleado insertado se convierte en la raíz del árbol.

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 2
Ingrese codigo a buscar: 70

Empleado encontrado:
Codigo: 70 | Nombre: Gerente Finanzas  | Cargo: Nodo interno
```

El árbol recorre los nodos comparando códigos hasta encontrar al empleado solicitado.

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 4

Recorrido Inorden:
Codigo: 20 | Nombre: Emp1 | Cargo: Hoja
Codigo: 30 | Nombre: Gerente de Ventas  | Cargo: Nodo interno
Codigo: 40 | Nombre: Emp2 | Cargo: Hoja
Codigo: 50 | Nombre: Empresa UTA | Cargo: Raiz
Codigo: 60 | Nombre: Emp3 | Cargo: Hoja
Codigo: 70 | Nombre: Gerente Finanzas  | Cargo: Nodo interno
Codigo: 80 | Nombre: Emp4 | Cargo: Hoja
```

Visita los nodos en orden: izquierda → raíz → derecha. Muestra los empleados ordenados de menor a mayor código.

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 5

Recorrido Preorden:
Codigo: 50 | Nombre: Empresa UTA | Cargo: Raiz
Codigo: 30 | Nombre: Gerente de Ventas  | Cargo: Nodo interno
Codigo: 20 | Nombre: Emp1 | Cargo: Hoja
Codigo: 40 | Nombre: Emp2 | Cargo: Hoja
Codigo: 70 | Nombre: Gerente Finanzas  | Cargo: Nodo interno
Codigo: 60 | Nombre: Emp3 | Cargo: Hoja
Codigo: 80 | Nombre: Emp4 | Cargo: Hoja
```

Visita los nodos en orden: raíz → izquierda → derecha. Empieza desde la raíz y desciende.

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 6

Recorrido Postorden:
Codigo: 20 | Nombre: Emp1 | Cargo: Hoja
Codigo: 40 | Nombre: Emp2 | Cargo: Hoja
Codigo: 30 | Nombre: Gerente de Ventas  | Cargo: Nodo interno
Codigo: 60 | Nombre: Emp3 | Cargo: Hoja
Codigo: 80 | Nombre: Emp4 | Cargo: Hoja
Codigo: 70 | Nombre: Gerente Finanzas  | Cargo: Nodo interno
Codigo: 50 | Nombre: Empresa UTA | Cargo: Raiz
```

Visita los nodos en orden: izquierda → derecha → raíz. Los nodos hoja aparecen primero.

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 7

Altura del arbol: 3
```

La altura del árbol es 3, lo que indica que hay 3 niveles desde la raíz hasta la hoja más lejana.

```
===== MENU ARBOL BST EMPRESARIAL =====
1. Insertar empleado
2. Buscar empleado
3. Mostrar raiz
4. Recorrido inorden
5. Recorrido preorden
6. Recorrido postorden
7. Mostrar altura
8. Mostrar hojas
0. Salir
Seleccione una opcion: 8

Nodos hoja:
Codigo: 20 | Nombre: Emp1 | Cargo: Hoja
Codigo: 40 | Nombre: Emp2 | Cargo: Hoja
Codigo: 60 | Nombre: Emp3 | Cargo: Hoja
Codigo: 80 | Nombre: Emp4 | Cargo: Hoja
```

Los nodos hoja son los que no tienen hijos: códigos 20, 40, 60 y 80.

Código	Nombre	Cargo
50	Empresa UTA	Raíz
30	Gerente Ventas	Nodo interno
70	Gerente Finanzas	Nodo interno
20	Emp 1	Hoja
40	Emp 2	Hoja
60	Emp 3	Hoja
80	Emp 4	Hoja

Datos de Prueba.

2.8 Habilidades blandas empleadas en la práctica

☐ Trabajo en equipo
☒ Responsabilidad

2.9 Conclusiones

- El Árbol Binario de Búsqueda (BST) demostró ser una estructura de datos eficiente para organizar información jerárquica, ya que al insertar los empleados usando el código numérico como clave, el árbol se organiza automáticamente permitiendo búsquedas con complejidad, lo que es significativamente más rápido que una búsqueda lineal en una lista simple.

- Los tres tipos de recorrido (inorden, preorden y postorden) tienen propósitos distintos y complementarios: el inorden resultó útil para mostrar los empleados ordenados por código, el preorden para visualizar la jerarquía desde la raíz, y el postorden para procesar primero los nodos hoja, demostrando la versatilidad del BST según la necesidad del sistema.

- La implementación en C++ usando recursividad en funciones como insertar(), buscar(), altura() y los recorridos, permitió escribir un código más limpio y comprensible, confirmando que la recursividad es la técnica natural para trabajar con árboles al aprovechar su estructura jerárquica y repetitiva.

2.10 Recomendaciones

- Se recomienda implementar la función de eliminación de nodos como mejora futura, ya que el programa actual no permite remover empleados del árbol. Esta operación es más compleja que la inserción porque debe considerar tres casos: eliminar una hoja, un nodo con un hijo, o un nodo con dos hijos.

- Para evitar que el árbol se desbalancee cuando se insertan datos en orden secuencial, se recomienda estudiar e implementar un Árbol AVL o Árbol Rojo-Negro, que son variantes del BST que se auto-balancean automáticamente.
- Se recomienda agregar validaciones de entrada al programa, como verificar que el código sea un número positivo, que el nombre no esté vacío, y manejar el caso en que el usuario ingrese letras donde se espera un número, para hacer el sistema más robusto y confiable en un entorno real.

2.11 Referencias bibliográficas

[1] "Definición del nodo raíz – Glosario | NordVPN." Accessed: Apr. 24, 2026. [Online]. Available: https://nordvpn.com/es/cybersecurity/glossary/root-node/

[2] "4 Types of Tree Traversal Algorithms | Built In." Accessed: Apr. 24, 2026. [Online]. Available: https://builtin.com/software-engineering-perspectives/tree-traversal

[3] "3.1 Árboles, explicación y ejemplos".

2.12 Anexos

Adjunto el link del repositorio en GitHub

https://github.com/Alexis112008/arbol-bst-empresa-cpp